JULY 2026

PROJECT REPORT

DESIGN AND DEVELOPMENT OF FMCW RADAR

Group Members:
Nihar.S
Preethi.A
Sri Harshan.H
Suhas.A

02

ABOUT PROJECT

This project focuses on the design and development of a real-time Frequency Modulated Continuous Wave (FMCW) radar signal processing pipeline. The objective is to detect static targets within 10–30 meters, estimate target range and velocity using Doppler analysis, and visualize results through range profiles and range-Doppler heatmaps. The system operates on IQ data and is implemented in C on a Linux platform.
The radar model employs linear FMCW chirps with a carrier frequency of 2.4 GHz, bandwidth of 40 MHz, and chirp duration of 1 ms.

Beat frequency extraction enables range estimation, while Doppler FFT provides velocity measurements. The software architecture is modular, comprising components for chirp configuration, FFT-based range and Doppler processing, windowing, CFAR detection, and visualization.
Validation is carried out in three phases: synthetic signal testing, static target experiments, and moving target detection, with performance metrics ensuring range accuracy within ±1.5 m. Deliverables include source code, design documentation, block diagrams, parameter justification, experimental results, and a Python-based visualization interface.
This work demonstrates the feasibility of implementing FMCW radar algorithms in real-time, providing a foundation for advanced radar applications in surveillance, automotive safety, and remote sensing.

03

KEY FEATURES

1. Realtime FMCW radar signal processing in C on Linux.
2. Detects static targets (10–30 m) and estimates range & velocity.
3. Uses 2.4 GHz carrier, 40 MHz bandwidth, 1 ms chirp.
4. Modular software design with FFTbased range/Doppler processing.
5. Outputs: range profile, rangeDoppler heatmap, target list.
6. Python interface for visualization.
7. Validated through synthetic, static, and moving target tests.

SYSTEM APPROACH

The proposed FMCW radar system is designed using a hardware–software co-design approach. The hardware section consists of an SDR-based radar front-end that captures real-world radar signals in the form of IQ samples. The software section is implemented in the C programming language on a Linux platform to process the acquired signals and estimate target range and velocity. The complete system follows a sequential processing pipeline consisting of signal acquisition, preprocessing, detection, and visualization.

The overall workflow of the system is:
Signal Acquisition → Signal Processing → Target Detection → Visualization

04

PROGRAM

Description : The Octave program reads the recorded FMCW radar signal for different delay values from binary data files. It converts the raw data into complex samples, performs an FFT on the signal, and identifies the peak frequency, which corresponds to the beat frequency. The detected beat frequency for each delay is displayed and plotted. Finally, the program generates a graph showing the relationship between delay and beat frequency, helping to visualize how the beat frequency changes as the signal delay increases.

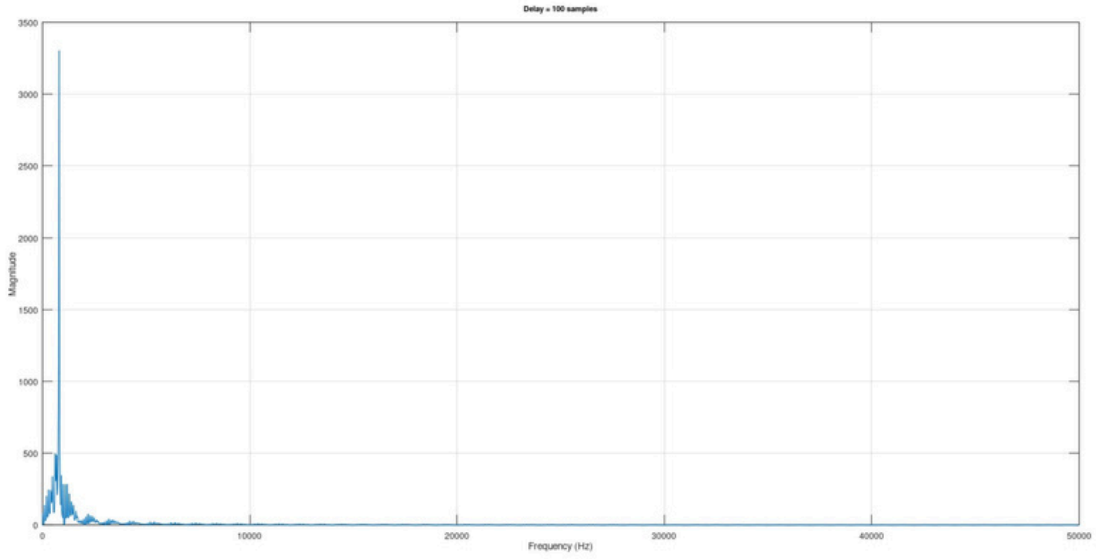
05

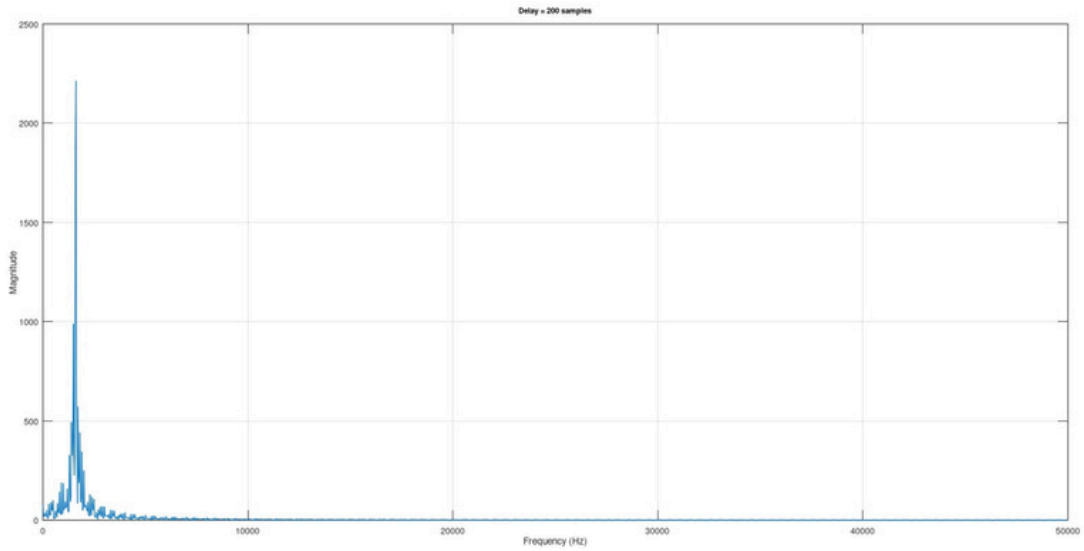
```
folder = "C:/Users/Preethi/Desktop/SDR/";
delays = [100 200];
Fs = 100000;

plot_enable = 1;

% Store beat frequencies
beat_freq = zeros(size(delays));

fprintf("\nDelay\tBeat Frequency (Hz)\n");

for k = 1:length(delays)
    delay = delays(k);
    filename = sprintf("%sbeat_%d.dat", folder, delay);
    fid = fopen(filename,"rb");
    if(fid==-1)
        fprintf("Cannot open %s\n",filename);
        continue;
    end
    data = fread(fid,200000,"float32");
    fclose(fid);

    x = data(1:2:end) + 1i*data(2:2:end);
    x_fft = x(1:4096);
    N = length(x_fft);
    X = abs(fft(x_fft));
    f = (0:N-1)*Fs/N;
    % Ignore DC
    [~,idx] = max(X(10:floor(N/2)));
    idx = idx + 9;
    beat_freq(k) = f(idx);
    fprintf("%d\t%.2f\n",delay,beat_freq(k));
    if(plot_enable)
        figure;
        plot(f(1:N/2),X(1:N/2));
        grid on;
        xlabel("Frequency (Hz)");
```

06

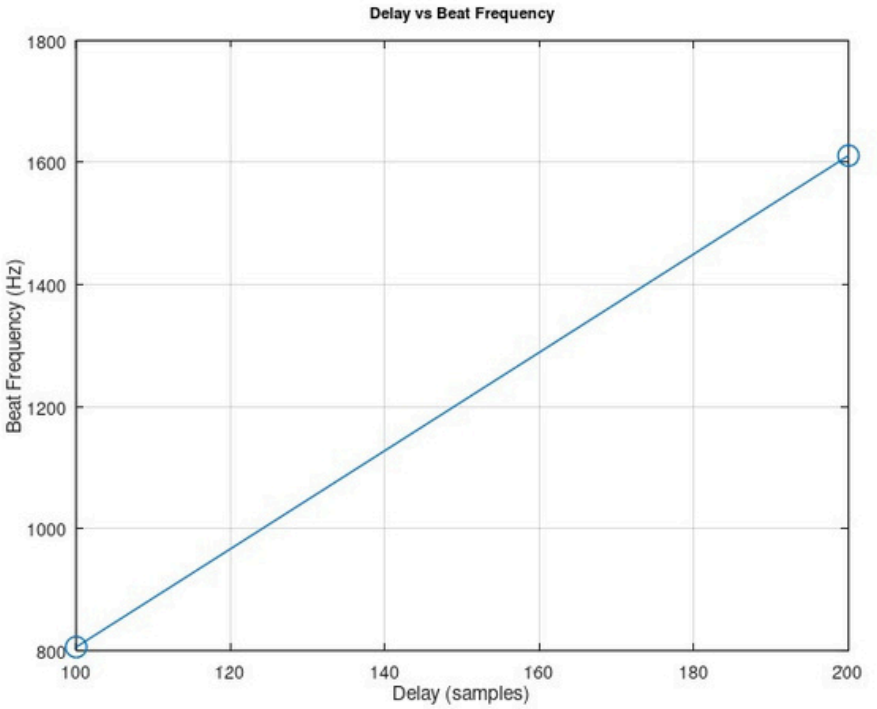
```
ylabel("Magnitude");
 title(sprintf("Delay = %d samples",delay));
 end
end
figure;
plot(delays,beat_freq,'-o');
grid on;
xlabel("Delay (samples)");
ylabel("Beat Frequency (Hz)")
title("Delay vs Beat Frequency");
```

OUTPUT:





07



CODE DESCRIPTION

- We extract the file from the GNU Radio using a file sink block.
- The code give the output of beat frequency for two different delays. i.e Delay = [100,200]

For doing this, we are using

- filename = sprintf("%sbeat_%d.dat, folder, delay);
- This takes the different files of the different delays that are saved in
- beat_100 or beat_200 format.
- X data(1:2:end) + 1idata (2:2:end); [this line is used to reconstruct the complex function.
- We are taking FFT for 4096 samples. 4096 is only taken particularly because it is power of 2 and it is easy to calculate the FFT for the powers of 2.

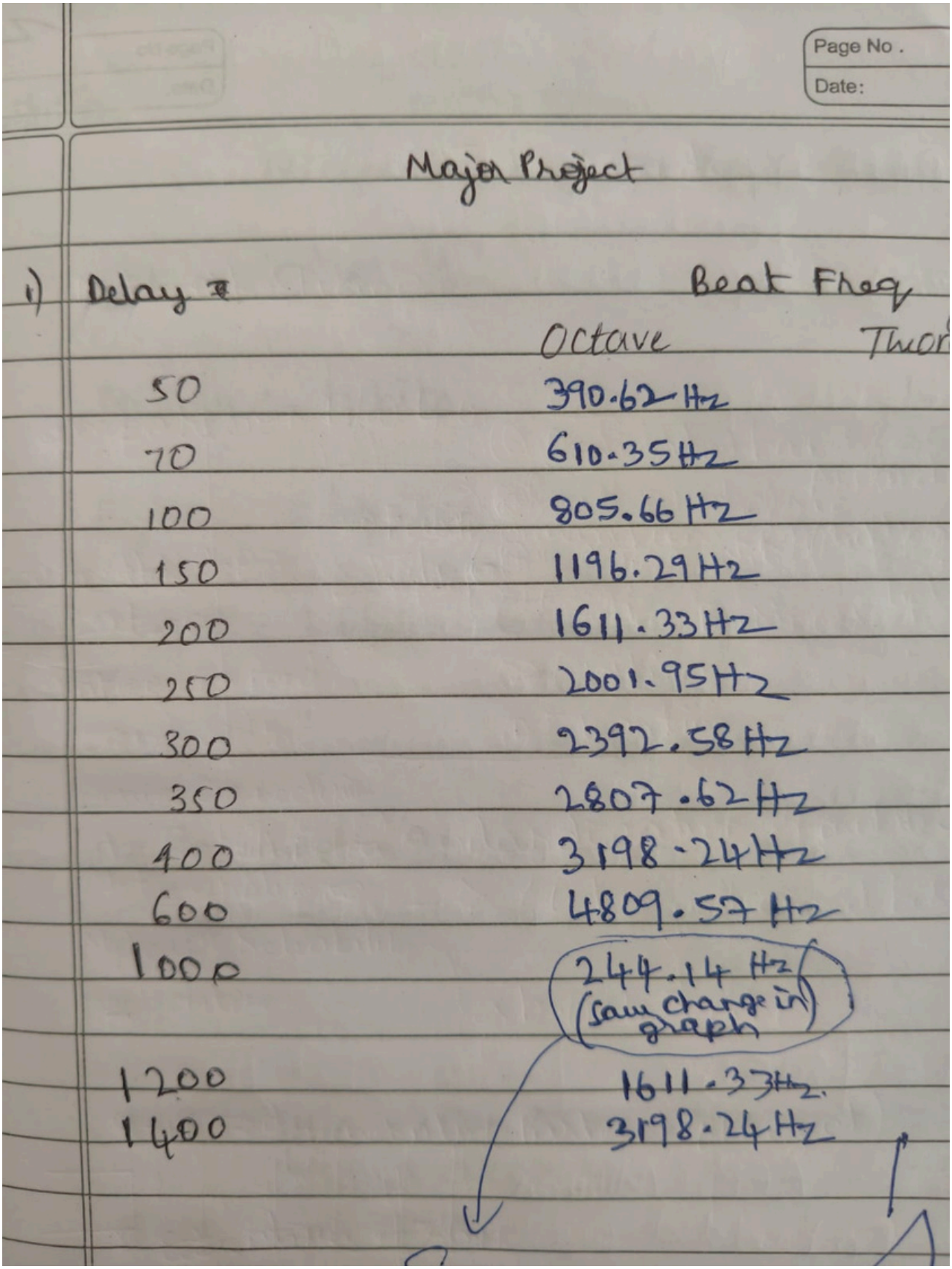
- [-,idx] = max(X(10:floor(N/2))); This line finds the highest peak of the frequency by ignoring the first peak because the first peak represents the DC component, which is not the actual Beat frequency. So, to not get any DC component peak, we are taking the index starting from 10th frequency value.

- idx idx + 9; as mentioned above this line is taking the 10th index

- beat_freq(k) = f(idx); shows the frequency corresponding to that FFT bin

- Finally the Beat frequency value is displayed

09



Page No .
Date:

Major Project

i) Delay ₹ Beat Freq

	Octave	Thor
50	390.62 Hz	
70	610.35 Hz	
100	805.66 Hz	
150	1196.29 Hz	
200	1611.33 Hz	
250	2001.95 Hz	
300	2392.58 Hz	
350	2807.62 Hz	
400	3198.24 Hz	
600	4809.57 Hz	
1000	244.14 Hz (Same change in graph)	
1200	1611.33 Hz.	
1400	3198.24 Hz	

10

* Beat Frequency Calculation

Signal Source :

Freq. = 100 Hz

Amp. = 10

offset = 2

Sawtooth varies from 2 to 12

$f_{VCO} = \frac{\text{Sensitivity}}{2\pi} \times x(t)$

Sensitivity = 5000 rad / sec

$f_{VCO} = \frac{5000}{2\pi} = 795.8 \text{ Hz}$

Sawtooth goes → 2 → 12

min freq = 2 × 795.8 = 1591 Hz.

max freq = 12 × 795.8 = 9549 Hz.

Sweep BW = 9549 - 1591 = 7958 Hz. ≅ 8KHz

Slope :- $T = \frac{1}{f} = \frac{1}{100} = 0.01 \text{ s}$.

$S = \frac{7958}{0.01} = 795800 \text{ Hz/s}$.

Delay :-

$\tau = \frac{\text{Delay}}{\text{samp rate}} = \frac{50}{100000} = 0.0005 \text{ sec}$

Beat freq :-

$f_b = S\tau$

$= 795800 \times 0.0005$

$f_b = 397.9 \text{ Hz}. \cong 398 \text{ Hz}.$